Practical Exercise - Engineering Manager/Lead Engineer

Practical Exercise — Engineering Manager/Lead Engineer

Role: Engineering Manager/Lead Engineer — a senior individual contributor, and the technical DRI for architecture, engineering execution and production quality in our neobank superapp.

Format: a time-boxed take-home. Roughly **120 minutes of work**, submitted as a **GitHub repository**, followed by a **30-minute review** in which we read it with you.

What we're assessing: how you take an open-ended applied-AI problem end to end — what you build, what you deliberately don't, how you know it works, and how well you defend those calls.

We've tried to make this brief specific enough that you can start without guessing. If something is still unclear, ask us — it won't count against you, and it usually means we didn't write it well in the first place.

The time box is part of the exercise

120 minutes is not enough to build everything below, and it isn't meant to be. We're not asking for a complete system. We're asking you to work out what a complete system would need, build the slice that proves the approach, and write down the rest.

A submission that ships a thin, correct, honestly-evaluated slice and says plainly what it doesn't do will score **above** one that sprawls. Building more than the problem needs counts against you here, not for you.

#	Section	Weight	Suggested time
1	Personal knowledge layer	25%	20 min
2	AI system and back-end architecture	25%	30 min
3	Evaluation, reliability and testing	30%	35 min
4	User experience	10%	15 min

5	Engineering leadership, decisions and AI use	10%	20 min

Environment setup and reading this brief do not count against the box. Time yourself honestly and record what you actually spent in your repo README. If you go over, say so — an accurate note costs you nothing, and we'd rather know. A twelve-hour submission doesn't beat a two-hour one that shows better judgement, and the commit history usually tells us which is which.

There is **no single right answer**, and we don't have a hidden one in mind. Where you need a fact we haven't given you, do write down your assumption and keep going.

The problem

Technical interviews open with the same questions: who are you, what have you built, what problems have you solved, how do you approach your work?

Build an **AI assistant that answers those questions on your behalf** — a queryable, grounded representation of your own professional background that a recruiter, hiring manager or interviewer could interact with to learn about your experience, projects, technical decisions and working style.

The subject is you, so the exercise is self-contained: no BJAK systems, no production data, nothing resembling unpaid work for us.

One constraint carries the weight of the role. **The assistant must not invent qualifications, employers, projects or achievements.** A system that fabricates an employer is the same failure class as one that fabricates a balance in financial industry.

We're AI-native and use AI heavily in how we build. We also hold the line that a model does not get to assert a fact nobody can trace back to a source. Show us how you hold that line in your own design.

Use whatever stack you think fits — commercial model APIs, open-source models, any vector store, search engine, framework, or none of the above. There is no expected stack, and a fashionable choice scores no better than a justified one.

Your tasks

Work through all five. Depth beats length. Every section is graded on your reasoning as much as on what runs.

1. Personal knowledge layer — 25%

Give the assistant something real to be grounded in.

Build a focused knowledge base from sources such as your CV, LinkedIn profile, technical writing, talks, GitHub repositories or project documentation, and anything else that reflects how you work. **Your CV is the minimum requirement.** You are not expected to build a polished integration for every source — show how you would collect, clean, structure, index and maintain this information.

Make clear in your repo:

- what information the assistant can see, and how you collected, cleaned, structured and indexed it,
- how it's stored, and how the system retrieves the most relevant context for a given question,
- how an additional source could be added later without redesigning what exists,
- **one real conflict, gap or stale fact in your own material** — an overlapping date range, a title that differs between CV and LinkedIn, an unfinished project — and exactly what the system does when a question lands on it,
- anything you redacted or replaced with synthetic content, and why.

No minimum public footprint is expected. A thin CV plus clearly-labelled synthetic material is a legitimate answer — we're grading the pipeline, not your career. This ships as a repository we'll read and you'll keep, so **commit only what you're comfortable having us read**. Redacting something costs you nothing, and neither does labelling synthetic content as synthetic.

2. AI system and back-end architecture — 25%

Design and implement the intelligence behind the assistant. Retrieval-augmented generation, an agent or tool-using workflow, a structured QA pipeline, a combination of retrieval, routing, prompting and deterministic logic — your call.

Your write-up should let a reviewer predict the system's behaviour without reading every line. Cover:

- **why this architecture** — and the simpler thing you considered and rejected, with what decided it,
- **how retrieval, prompting and any tool use actually work**, concretely,
- **how a stated fact is traced back to its source**, and what the system does when it can't be,
- **model choice**, and what evidence would make you switch,
- **failure paths** — the model API is slow, down, rate-limited, or returns something unusable; what the user sees; what the deterministic fallback is,
- **security and privacy** — where secrets live, what data leaves your machine, what a provider may retain or train on,

- a **rough cost per question and a measured p95 latency**; two real numbers beat a paragraph about scalability,
- **what you would change before this went to production**, and what breaks first at 100× the traffic.

The repository should take its configuration from environment variables and ship an `.env.example` naming each one. **Please don't commit a real key** — if one slips in, rotate it and tell us; we'd far rather hear it from you.

3. Evaluation, reliability and testing — 30%

The heaviest section, deliberately. Where being wrong is expensive, "it seemed to work when I tried it" is not evidence.

Build an evaluation you actually ran, and commit it:

- **An evaluation dataset** — at least **20** questions an interviewer or recruiter might realistically ask, each labelled with its category and with what a correct answer must contain or must never claim. Include all five of: answerable directly from the CV; requiring two or more sources combined; ambiguous or underspecified; **not answerable from the knowledge base**, where the assistant must say so; and **adversarial** — attempts to make it exaggerate or fabricate your experience, step outside scope, or follow an injected instruction.
- **A runnable evaluation** — one documented command that scores the dataset and prints a results table.
- **The results you actually got**, committed, with a stated pass bar and an error analysis of at least two failures. A submission reporting 72% groundedness with an honest account of where it broke scores above one reporting 100% with no method.
- **Metrics you define precisely** — pick a few that matter (retrieval relevance, answer correctness, groundedness or citation accuracy, hallucination rate, refusal correctness on unanswerable questions, consistency across reworded questions, latency) and state the numerator, denominator and what counts as a pass. A metric nobody can recompute is not a metric.
- If you use **LLM-as-judge**, commit the judge prompt and name the safeguard — spot-check a sample against your own labels and report the agreement.

Tell us what you would add to this evaluation if it were guarding money movement rather than a CV, and how it would run continuously rather than once.

4. User experience — 10%

Provide a usable way to interact with the assistant. A command-line application, a notebook, a small web app, a chat UI — a basic interface is fine if the system behind it is well designed. We'd rather see a CLI that shows its sources than a polished chat window that doesn't.

Two behaviours must be visible in whatever you build:

- the **sources** behind an answer, and
- a **clear fallback** when the assistant can't answer from what it knows.

Anything else — suggested questions, conversation history, streaming, confidence indicators, a tone that sounds like you — is optional. Add one line on the interaction decision you made and who it serves.

5. Engineering leadership, decisions and AI use — 10%

This section is writing, not building. Keep it short and specific.

- **A decision record** — three to five decisions you made, each with the alternative you rejected and the constraint or evidence that decided it. A few lines each.
- **What you cut**, why it was the right thing to cut inside the time box, and the next three things you would do.
- **Where you used AI, and where you deliberately did not** — which parts were model-generated, what you changed after reviewing them, and one thing you refused to let a model decide. Using AI heavily is expected here; not being able to account for what it produced is the problem.
- **A self-review** — the two changes you would ask for if this arrived as someone else's pull request.

Submission

Please send us a **GitHub repository** rather than a ZIP file or an email attachment — we want to read the history as well as the code.

1. **Repository name:** `bjak-lead-engineer-<yourname>`, lowercase and hyphenated.
2. **Visibility:** public, or private. Private is the better choice if the repo carries personal data — say which you chose when you send the link, and if it's private, add **wonglingyit@bjak.my **as the collaborator for review purpose.
3. **A root `README.md` that is the entry point.** Another engineer should be able to understand and run your project from it with no further guidance. It must cover: what this is; how to install and run it in under ten minutes; the architecture and why; the evaluation approach and the results you got; known limitations and what you would do next; your AI-usage note; and the time you spent.
4. **We need to be able to run it.** One documented command path, pinned dependencies, an `.env.example` naming every variable, and no committed secrets. If we'll need an API key of our own, tell us which provider and roughly what one evaluation run costs.

5. **Commit the knowledge sources** — real, redacted or synthetic — so the system is actually runnable, along with your evaluation dataset, evaluation script and committed results.
6. **Work in real commits.** A sequence of commits with meaningful messages is part of the submission, and a single "initial commit" containing a finished system tells us much less. The history is genuinely useful to us — it shows how you worked, not just what you landed on.
7. **Declare anything you did not write for this exercise** — a template, a starter, or a prior project you built on. Reuse is fine; undisclosed reuse is not.
8. **Tag the commit you want reviewed:** `git tag submission && git push --tags`, so later pushes don't confuse the review.

When it's ready, reply to your assessment email with the repository URL, saying whether it's public or private, to:

wonglingyit@bjak.my **and cc: **recruitment@bjak.my

Please submit within **5 calendar days** of receiving this brief. If that window doesn't work for you, tell us and we'll move it — we'd rather have your judgement than your availability.

What we're looking for

We're not expecting a polished production system. We're looking for practical applied engineering ability, structured problem-solving under a real constraint, and sensible architectural decisions. Alongside those: strong coding and documentation practice, a clear-eyed view of what models get wrong, a serious approach to testing and evaluation, and the judgement to turn an open-ended problem into something usable.

What will not score well: a system whose behaviour you can't explain, an evaluation written after the results, a complete-looking build with no account of how it fails, and complexity a simpler design would have covered.

What happens next

Your repository is the starting point for the next conversation, not the end of the assessment. In a **30-minute review** we'll read the code with you, run your evaluation, and push on the decisions behind both — including the ones you cut. Expect to make a small change live.

We enjoy this conversation most when someone brings the *why*, so come ready to disagree with us. Good luck — we're looking forward to reading it.